



let tasks = [

```
{ id: 't1', title: 'Design System', status: 'todo', tag: 'Design' },  
{ id: 't2', title: 'API Integration', status: 'inprogress', tag: 'Dev' },  
{ id: 't3', title: 'Client Feedback', status: 'done', tag: 'Urgent' }
```

];

```
document.addEventListener('DOMContentLoaded', () => { renderBoard(); renderStaticData(); });
```

```
function switchView(viewId) {  
  document.querySelectorAll('.view').forEach(el => el.classList.remove('active-view', 'hidden-view'));  
  document.querySelectorAll('.view').forEach(el => el.classList.add('hidden-view'));  
  const active = document.getElementById(viewId);  
  active.classList.remove('hidden-view');  
  active.classList.add('active-view');
```

```
  document.querySelectorAll('.nav-links li').forEach(li => li.classList.remove('active'));  
  event.currentTarget.classList.add('active');
```

}

```
function renderBoard() {  
  ['todo', 'inprogress', 'done'].forEach(status => {  
    const filtered = tasks.filter(t => t.status === status);  
    document.getElementById(count-`${status}`).innerText = filtered.length;
```

```
    const html = filtered.map(task => `  
      <div class="task-card" draggable="true" id="${task.id}"  
        ondragstart="drag(event)" ondragend="dragEnd(event)">  
        <strong>${task.title}</strong>  
        <small>${task.tag}</small>  
      </div>  
    `).join('');  
  
    document.getElementById(`list-${status}`).innerHTML = html;  
  });
```

}

```

function allowDrop(ev) { ev.preventDefault(); }
function drag(ev) {
  ev.dataTransfer.setData("text", ev.target.id);
  ev.target.classList.add('dragging');
}
function dragEnd(ev) { ev.target.classList.remove('dragging'); }

```

```

function drop(ev) {
  ev.preventDefault();
  const taskId = ev.dataTransfer.getData("text");
  const column = ev.target.closest('.column');
  if (!column) return;

```

```

  const task = tasks.find(t => t.id === taskId);
  if (task && task.status !== column.id) {
    task.status = column.id;
    renderBoard();
  }

```

```

}

```

```

function openModal() { document.getElementById("taskModal").style.display = "flex"; }
function closeModal() { document.getElementById("taskModal").style.display = "none"; }

```

```

function addTask(e) {
  e.preventDefault();
  const title = document.getElementById('taskTitle').value;
  if(!title) return;

```

```

  tasks.push({
    id: 't' + Date.now(),
    title: title,
    status: 'todo',
    tag: document.getElementById('taskTag').value
  });

```

```

  renderBoard();
  closeModal();
  document.getElementById('taskForm').reset();

```

```

}

```

```

function renderStaticData() {
  const products = [
    { name: 'Leather Satchel', sku: 'L-01', origin: 'Italy' },
    { name: 'Canvas Tote', sku: 'C-09', origin: 'India' }
  ];
  document.getElementById('product-table-body').innerHTML = products.map(p =>

```

```
<tr><td>${p.name}</td><td>${p.sku}</td><td>${p.origin}</td></tr>
).join('');
```

```
const repos = [
  { name: 'Base-Kit', lang: 'HTML/CSS' },
  { name: 'Server-Node', lang: 'JavaScript' }
];
document.getElementById('repo-grid').innerHTML = repos.map(r =>
  `
    <div class="repo-card"><h3>${r.name}</h3><p>${r.lang}</p></div>`
).join('');
```

```
}
```

explain the whole code to a first timer

Now let me break this down for you step by step!

Big Picture: What Does This Code Do?

This is a **Kanban board** (like Trello) with three columns: "To Do", "In Progress", and "Done". You can drag tasks between columns, add new tasks, and see data in a table and repository cards. Think of it as a project management tool.

Part 1: The Data (Tasks Array)

```
let tasks = [
  { id: 't1', title: 'Design System', status: 'todo', tag: 'Design' },
  { id: 't2', title: 'API Integration', status: 'inprogress', tag: 'Dev' },
  { id: 't3', title: 'Client Feedback', status: 'done', tag: 'Urgent' }
];
```

What's happening:

- You have a list of **objects** (each task is an object)
- Each task has 4 properties:
 - `id` - unique identifier (like t1, t2, t3)
 - `title` - what the task is called
 - `status` - where it is (todo, inprogress, or done)
 - `tag` - category label

Think of it like a spreadsheet where each row is a task.

Part 2: Page Setup (DOMContentLoaded)

```
document.addEventListener('DOMContentLoaded', () => {
  renderBoard();
  renderStaticData();
});
```

What's happening:

- DOMContentLoaded = "Hey JavaScript, wait until the HTML page is fully loaded"
- Then run two functions:
 1. `renderBoard()` - display the Kanban board
 2. `renderStaticData()` - display the products and repositories

Why wait? Because you can't display something on a page if that element doesn't exist yet!

Part 3: Switching Views

```
function switchView(viewId) {
  // Hide all views
  document.querySelectorAll('.view').forEach(el => el.classList.remove('active-view', '
  document.querySelectorAll('.view').forEach(el => el.classList.add('hidden-view'));

  // Show the selected view
  const active = document.getElementById(viewId);
  active.classList.remove('hidden-view');
  active.classList.add('active-view');

  // Update navigation (which tab is selected)
  document.querySelectorAll('.nav-links li').forEach(li => li.classList.remove('active'
  event.currentTarget.classList.add('active'));
}
```

What's happening:

- This lets you switch between different screens (like tabs)
- First, hide everything by adding `'hidden-view'` class
- Then show only the view you clicked on
- Update the navigation to show which tab is active

Real-world analogy: Like clicking between "Board", "Products", and "Repos" tabs.

Part 4: Rendering the Board (Main Function!)

```
function renderBoard() {
  ['todo', 'inprogress', 'done'].forEach(status => {
    // Filter tasks by status
    const filtered = tasks.filter(t => t.status === status);

    // Show count of tasks
    document.getElementById(`count-${status}`).innerText = filtered.length;

    // Create HTML for each task
    const html = filtered.map(task => `
      <div class="task-card" draggable="true" id="${task.id}"
        ondragstart="drag(event)" ondragend="dragEnd(event)">
        <strong>${task.title}</strong>
        <small>${task.tag}</small>
      </div>
    `).join('');

    // Put HTML on the page
    document.getElementById(`list-${status}`).innerHTML = html;
  });
}
```

What's happening (step-by-step):

1. **Loop through 3 statuses:** ['todo', 'inprogress', 'done']
2. **For each status:**
 - `filter()` → Get only tasks with that status
 - Update the count (e.g., "3 To Do tasks")
 - `map()` → Transform each task into HTML code
 - `join('')` → Combine all HTML into one string
 - Put that HTML on the page with `innerHTML`

Key concepts:

- `.filter()` - keeps only items that match a condition
- `.map()` - transforms each item (task object → HTML)
- **Template literals** (``${variable}``) - insert JavaScript into text
- `draggable="true"` - makes the card draggable

Part 5: Drag and Drop

```
function allowDrop(ev) {
  ev.preventDefault();
}

function drag(ev) {
```

```

    ev.dataTransfer.setData("text", ev.target.id);
    ev.target.classList.add('dragging');
  }

function dragEnd(ev) {
  ev.target.classList.remove('dragging');
}

function drop(ev) {
  ev.preventDefault();
  const taskId = ev.dataTransfer.getData("text");
  const column = ev.target.closest('.column');
  if (!column) return;

  const task = tasks.find(t => t.id === taskId);
  if (task && task.status !== column.id) {
    task.status = column.id;
    renderBoard();
  }
}

```

What's happening:

Function	Purpose
allowDrop(ev)	Allows items to be dropped here (required!)
drag(ev)	When you start dragging, save the task ID and add "dragging" style
dragEnd(ev)	When you stop dragging, remove the "dragging" style
drop(ev)	When you drop: find the task, change its status, re-render

Real example:

1. You drag "Design System" (t1)
2. `drag()` saves "t1" in memory
3. You drop it in the "Done" column
4. `drop()` finds the task, changes status from 'todo' to 'done'
5. `renderBoard()` updates the display

Part 6: Modal (Pop-up for Adding Tasks)

```

function openModal() {
  document.getElementById("taskModal").style.display = "flex";
}

function closeModal() {
  document.getElementById("taskModal").style.display = "none";
}

function addTask(e) {

```

```

e.preventDefault();
const title = document.getElementById('taskTitle').value;
if(!title) return;

tasks.push({
  id: 't' + Date.now(),
  title: title,
  status: 'todo',
  tag: document.getElementById('taskTag').value
});

renderBoard();
closeModal();
document.getElementById('taskForm').reset();
}

```

What's happening:

- `openModal()` - Show the pop-up form
- `closeModal()` - Hide the pop-up form
- `addTask()` - When you submit the form:
 1. Get the title and tag from input fields
 2. Create a new task object with unique ID (using `Date.now()`)
 3. Add it to the `tasks` array with `push()`
 4. Re-render the board
 5. Close the modal
 6. Clear the form

Part 7: Rendering Static Data

```

function renderStaticData() {
  const products = [
    { name: 'Leather Satchel', sku: 'L-01', origin: 'Italy' },
    { name: 'Canvas Tote', sku: 'C-09', origin: 'India' }
  ];
  document.getElementById('product-table-body').innerHTML = products.map(p =>
    `<tr><td>${p.name}</td><td>${p.sku}</td><td>${p.origin}</td></tr>`
  ).join('');

  const repos = [
    { name: 'Base-Kit', lang: 'HTML/CSS' },
    { name: 'Server-Node', lang: 'JavaScript' }
  ];
  document.getElementById('repo-grid').innerHTML = repos.map(r =>
    `<div class="repo-card"><h3>${r.name}</h3><p>${r.lang}</p></div>`
  ).join('');
}

```

What's happening:

1. Products table:

- Create array of product objects
- Use `map()` to convert each into a table row (`<tr>`)
- Put the HTML in the page

2. Repos grid:

- Create array of repo objects
- Use `map()` to convert each into a card div
- Put the HTML in the page

Both use the same pattern: data → HTML → display

Key Concepts Summary

Concept	What It Does
<code>.filter()</code>	Keep only items matching a condition
<code>.map()</code>	Transform each item into something new
<code>.find()</code>	Get the first item matching a condition
<code>.push()</code>	Add item to end of array
<code>innerHTML</code>	Replace all HTML inside an element
<code>classList</code>	Add/remove CSS classes (for styling)
Event listeners	React when user does something (drag, click)
Template literals	Write text with variables: <code>`Hello \${name}`</code>

Quick Visual Flow

Page loads



DOMContentLoaded triggers



`renderBoard()` → Display tasks in 3 columns

`renderStaticData()` → Display products & repos



User interacts:

- ├ Drag task → `drop()` → update status → `renderBoard()`
- ├ Click "Add" → `openModal()` → `addTask()` → `renderBoard()`
- └ Click tab → `switchView()` → show different section

Hope this helps! Let me know if you want me to explain any specific part in more detail! ☺



1. https://download.microsoft.com/download/2/6/e/26ef0117-c634-456c-878e-166c98c7c266/mdn_0114dg.pdf
2. https://www.w3schools.com/html/html5_draganddrop.asp
3. <https://chromium.googlesource.com/chromium/src/+ac6b00ded1..d66554b28a>
4. <https://www.w3.org/TR/2006/WD-WCAG20-TECHS-20060427/>
5. <https://docs.oracle.com/en/cloud/paas/mobile-hub/develop/developing-applications-oracle-mobile-hub.pdf>
6. https://djp3.westmont.edu/classes/2016_09_CS010/Lectures/Lecture_37/output.txt
7. https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API
8. <https://www.freecodecamp.org/news/dom-manipulation-in-javascript/>
9. <https://stackoverflow.com/questions/34398279/map-and-filter-an-array-at-the-same-time>
10. <https://raw.githubusercontent.com/whatwg/html/master/source>